FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

LABEXER1

Sison, Xanneder S,	Mr. Hadji Tejuco
Name of Student	Name of Professor
10/16/2026	10/16/2026
Data Performed	Date Submitted

Source Code

```
; LABEXER1 - NASM 32-bit Linux
; Arithmetic menu using procedures

section .data
    menu db 10, "------------------------------", 10
         db "MENU", 10
         db "[A]dd", 10
         db "[S]ubtract", 10
         db "[M]ultiply", 10
         db "[D]ivide", 10
         db "------------------------------", 10, 0

    choicePrompt db "Enter your choice: ", 0
    firstPrompt  db "Enter first number (0-99): ", 0
    secondPrompt db "Enter second number (0-99): ", 0

    sumMsg        db "The sum is ", 0
    differenceMsg db "The difference is ", 0
    productMsg    db "The product is ", 0
    quotientMsg   db "The quotient is ", 0
    remainderMsg  db "The remainder is ", 0

    invalidChoice db "Invalid choice. Enter A, S, M, or D.", 10, 0
    invalidNumber db "Invalid input. Enter a number from 0 to 99.", 10, 0
    divideZeroMsg db "Cannot divide by zero.", 10, 0

    againPrompt db 10, "Do you want to try again (Y/N)? ", 0
    invalidAgain db "Please enter Y or N.", 10, 0

    clearScreen db 27, "[2J", 27, "[H", 0
    minusSign   db "-", 0
    newLine     db 10, 0

section .bss
    inputBuffer resb 32
    inputChar   resb 1
    number1     resd 1
    number2     resd 1
    digitBuffer resb 12

section .text
    global _start

_start:

menu_loop:
    mov ecx, menu
    call print_string

    mov ecx, choicePrompt
    call print_string
    call read_line

    cmp eax, 1
    jne invalid_choice

    mov al, [inputBuffer]
    and al, 0DFh                ; Convert lowercase to uppercase

    cmp al, 'A'
    je addition

    cmp al, 'S'
    je subtraction
```

```
    cmp al, 'M'
    je multiplication

    cmp al, 'D'
    je division

invalid_choice:
    mov ecx, invalidChoice
    call print_string
    jmp menu_loop

addition:
    call get_numbers
    call add_numbers

    mov ecx, sumMsg
    call print_result
    jmp ask_again

subtraction:
    call get_numbers
    call subtract_numbers

    mov ecx, differenceMsg
    call print_result
    jmp ask_again

multiplication:
    call get_numbers
    call multiply_numbers

    mov ecx, productMsg
    call print_result
    jmp ask_again

division:
    call get_numbers

    cmp dword [number2], 0
    je division_by_zero

    call divide_numbers

    ; Save remainder while displaying the quotient
    push edx

    mov ecx, quotientMsg
    call print_result

    pop eax
    mov ecx, remainderMsg
    call print_result
    jmp ask_again

division_by_zero:
    mov ecx, divideZeroMsg
    call print_string

ask_again:
    mov ecx, againPrompt
    call print_string
    call read_line

    cmp eax, 1
    jne invalid_again

    mov al, [inputBuffer]
    and al, 0DFh
```

```
    cmp al, 'Y'
    je restart

    cmp al, 'N'
    je exit_program

invalid_again:
    mov ecx, invalidAgain
    call print_string
    jmp ask_again

restart:
    mov ecx, clearScreen
    call print_string
    jmp menu_loop

; ---------------------------------------
; Procedure: get_numbers
; Reads and stores the two operands
; ---------------------------------------
get_numbers:
    mov ecx, firstPrompt
    call read_number
    mov [number1], eax

    mov ecx, secondPrompt
    call read_number
    mov [number2], eax
    ret

; ---------------------------------------
; Arithmetic procedures
; Return the result in EAX
; ---------------------------------------
add_numbers:
    mov eax, [number1]
    add eax, [number2]
    ret

subtract_numbers:
    mov eax, [number1]
    sub eax, [number2]
    ret

multiply_numbers:
    mov eax, [number1]
    imul eax, [number2]
    ret

divide_numbers:
    mov eax, [number1]
    xor edx, edx
    div dword [number2]
    ; EAX = quotient, EDX = remainder
    ret

; ---------------------------------------
; Procedure: read_number
; ECX = prompt address
; Returns EAX = validated number from 0-99
; ---------------------------------------
read_number:
    push ecx
```

```
.retry:
    mov ecx, [esp]
    call print_string
    call read_line

    cmp eax, 1
    jb .invalid
    cmp eax, 2
    ja .invalid

    mov edx, eax                ; Number of digits
    xor eax, eax                ; Accumulated value
    xor edi, edi                ; Character index
.convert:
    movzx ebx, byte [inputBuffer + edi]

    cmp ebx, '0'
    jb .invalid
    cmp ebx, '9'
    ja .invalid

    sub ebx, '0'
    imul eax, 10
    add eax, ebx

    inc edi
    cmp edi, edx
    jb .convert

    pop ecx
    ret
.invalid:
    mov ecx, invalidNumber
    call print_string
    jmp .retry
; ----------------------------------------
; Procedure: read_line
; Reads one line into inputBuffer
; Returns EAX = number of characters
; ----------------------------------------
read_line:
    push ebx
    push ecx
    push edx
    push edi

    xor edi, edi
.next_character:
    mov eax, 3                  ; sys_read
    mov ebx, 0                  ; Standard input
    mov ecx, inputChar
    mov edx, 1
    int 80h

    cmp eax, 0
    jle .end_of_input

    mov al, [inputChar]

    cmp al, 10                  ; Enter / line feed
    je .done
```

```
    cmp al, 13                  ; Ignore carriage return
    je .next_character

    cmp edi, 31
    jae .too_long

    mov [inputBuffer + edi], al
    inc edi
    jmp .next_character

.too_long:
    mov edi, 32
    jmp .next_character

.end_of_input:
    test edi, edi
    jz exit_program

.done:
    mov eax, edi

    pop edi
    pop edx
    pop ecx
    pop ebx
    ret

; ----------------------------------------
; Procedure: print_result
; ECX = result message, EAX = result
; ----------------------------------------
print_result:
    call print_string
    call print_number

    mov ecx, newLine
    call print_string
    ret

; ----------------------------------------
; Procedure: print_number
; Prints the signed integer in EAX
; ----------------------------------------
print_number:
    pushad

    test eax, eax
    jns .positive

    mov ecx, minusSign
    call print_string
    neg eax

.positive:
    mov edi, digitBuffer + 11
    mov byte [edi], 0
    mov ebx, 10

.convert_digit:
    xor edx, edx
    div ebx

    add dl, '0'
    dec edi
    mov [edi], dl
```

```
    test eax, eax
    jnz .convert_digit

    mov ecx, edi
    call print_string

    popad
    ret

; ----------------------------------------
; Procedure: print_string
; ECX = address of a zero-terminated string
; Preserves all general-purpose registers
; ----------------------------------------
print_string:
    pushad
    xor edx, edx

.find_length:
    cmp byte [ecx + edx], 0
    je .write
    inc edx
    jmp .find_length

.write:
    mov eax, 4                  ; sys_write
    mov ebx, 1                  ; Standard output
    int 80h

    popad
    ret

exit_program:
    mov eax, 1                  ; sys_exit
    xor ebx, ebx
    int 80h
```

Program Explanation:

The program displays a menu for addition, subtraction, multiplication, and division. The get_numbers procedure reads two numbers from 0 to 99. The read_number procedure checks the input and converts the digits into an integer. Separate arithmetic procedures calculate the selected result, while print_number converts the result into decimal text.

Subtraction can display a negative result. Division displays an integer quotient and remainder, and checks for a zero divisor before dividing. After each operation, Y clears the screen and returns to the menu; N terminates the program. Invalid menu choices and invalid Y/N answers are requested again.

Compilation Process:

```
PS C:\Users\Calix\Documents\ComputerSystems\hjtnasm> docker compose run --rm nasm

Container hjtnasm-nasm-run-971f9b85121f Creating
Container hjtnasm-nasm-run-971f9b85121f Created
root@19a4d2fdb015:/work# cd /work/LABEXER1
root@19a4d2fdb015:/work/LABEXER1# nasm -f elf32 LABEXER1.asm -o LABEXER1.o
root@19a4d2fdb015:/work/LABEXER1# ld -m elf_i386 LABEXER1.o -o LABEXER1
root@19a4d2fdb015:/work/LABEXER1# file LABEXER1
LABEXER1: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically linked, not stripped
root@19a4d2fdb015:/work/LABEXER1# ./LABEXER1

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice:
```

Addition Test Cases

```
-----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
-----------------------------
Enter your choice: A
Enter first number (0-99): 2
Enter second number (0-99): 3
The sum is 5

Do you want to try again (Y/N)? Y

-----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
-----------------------------
Enter your choice: A
Enter first number (0-99): 12
Enter second number (0-99): 34
The sum is 46

Do you want to try again (Y/N)? Y

-----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
-----------------------------
Enter your choice: A
Enter first number (0-99): 99
Enter second number (0-99): 99
The sum is 198

Do you want to try again (Y/N)? Y
```

Subtraction Test Cases

```
------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: S
Enter first number (0-99): 9
Enter second number (0-99): 4
The difference is 5

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: S
Enter first number (0-99): 4
Enter second number (0-99): 9
The difference is -5

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: S
Enter first number (0-99): 88
Enter second number (0-99): 29
The difference is 59

Do you want to try again (Y/N)? Y
```

Multiplication Test Cases

```
------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: M
Enter first number (0-99): 3
Enter second number (0-99): 4
The product is 12

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: 12
Invalid choice. Enter A, S, M, or D.

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: M
Enter first number (0-99): 12
Enter second number (0-99): 12
The product is 144

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: M
Enter first number (0-99): 99
Enter second number (0-99): 99
The product is 9801

Do you want to try again (Y/N)? Y
```

Division Test Cases

```
Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: D
Enter first number (0-99): 8
Enter second number (0-99): 2
The quotient is 4
The remainder is 0

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: D
Enter first number (0-99): 7
Enter second number (0-99): 2
The quotient is 3
The remainder is 1

Do you want to try again (Y/N)? D
Please enter Y or N.

Do you want to try again (Y/N)? Y

------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: D
Enter first number (0-99): 12
Enter second number (0-99): 0
Cannot divide by zero.

Do you want to try again (Y/N)? N
root@8808137965ea:/work/LABEXER1#
```